

MANIMA

Manim Animation MCP Application Software Architecture Document

arc42 • Version 0.1 • Draft for Review
Cyril • 13 July 2026

Table of Contents

1. Introduction and Goals

MANIMA is a local-first Model Context Protocol (MCP) server that turns mathematical animation requests into rendered Manim Community Edition videos. It exposes two tool surfaces: a thin renderer that executes Manim source supplied by the calling client, and a thick generator that produces Manim source locally using Apertus 8B served by vLLM, without any code or prompt leaving the workstation.

The product goal is animation. A secondary and deliberately external goal is that MANIMA constitutes a realistic workload against which Apertus can be exercised under load — but that is measured around the application, not built into it. No architectural decision here is made to serve the benchmark at the animation tool's expense.

1.1 Requirements Overview

ID	Requirement	Priority
FR-1	Render supplied Manim CE source to video within a sandbox	Must
FR-2	Generate Manim CE source from a natural-language prompt using a local model	Must
FR-3	Verify generated source by probe-rendering; repair on failure within a bounded budget	Must
FR-4	Expose all long-running work through an asynchronous job protocol	Must
FR-5	Return artifacts as filesystem paths, never as inline binary payloads	Must
FR-6	Support full LaTeX typesetting in scenes	Must
FR-7	Permit opt-in escalation to a hosted model after local repair is exhausted	Should
FR-8	Ground generation in a retrievable Manim CE API corpus	Should
FR-9	Allow cancellation of an in-flight job	Should

1.2 Quality Goals

Priority	Quality Goal	Concrete Scenario
1	Security	A generated scene containing <code>os.system('rm -rf ~')</code> is either rejected by static validation or contained by the sandbox, with no effect on the host.
2	Sovereignty	With the escalation gate closed (default), no prompt, source, or artifact leaves the workstation. Verifiable by running with the host network disabled.
3	Correctness	A generated scene that reaches <code>SUCCEEDED</code> renders without traceback at the requested quality — verified, not assumed.
4	Responsiveness	No MCP tool call blocks for more than 2 seconds. Render duration is decoupled from call duration.
5	Observability	For any completed job it is recoverable which generator produced the source, how many repair attempts were consumed, and whether escalation occurred.

1.3 Stakeholders

Role	Expectation
Operator (Cyril)	Runs MANIMA on the workstation; expects local-first operation, explicit control over escalation, and honest reporting of generation quality.
MCP Client (Claude)	Calls the tools; expects a stable async contract and machine-readable failure detail rather than opaque errors.
Downstream consumer	Receives rendered artifacts for teaching material, talks, or documentation.

2. Architecture Constraints

Constraint	Description	Consequence
C-1 Local-first	Inference and rendering run on the operator's workstation. Network egress is denied by default.	Escalation must be an explicit, gated, logged exception — not a silent fallback.
C-2 Untrusted code	Manim renders by executing arbitrary Python. Generated code is untrusted by construction.	Execution must be sandboxed unconditionally, including for operator-supplied source.
C-3 MCP transport	MCP tool calls are synchronous request/response over stdio and carry a modest payload budget.	Long renders require an async job protocol; artifacts must be referenced, not embedded.
C-4 Manim CE API drift	Manim CE's API has changed materially across versions, and ManimGL forms are widely present in training data.	Generation must be grounded and verified; an ungrounded generator will emit plausible, non-rendering code.
C-5 TeX footprint	Full LaTeX support requires a TeX distribution in the render image (multi-gigabyte).	Image size is accepted as a cost; the image is built once and cached.
C-6 Language	Manim CE is Python. The MCP server may be Python or TypeScript.	Selecting Python for the server removes a process boundary but couples the server to the render runtime's ecosystem.
C-7 Windows host	The workstation runs Windows 11. Both vLLM and Linux containers require WSL2.	The entire server side lives in the WSL2 VM. Sandbox guarantees are those of WSL2 + Docker Desktop, which are weaker than Linux-native rootless Docker.
C-8 16 GB VRAM	The GPU has 16 GB. Apertus 8B at int4 fits (~6 GB); 70B does not (~40 GB at Q4).	The generator is 8B. This is arithmetic, not preference, and it caps achievable generation quality.

3. Context and Scope

MANIMA sits entirely inside the workstation trust boundary. The only component outside it is the escalation LLM, which is unreachable unless the operator explicitly opens the gate.

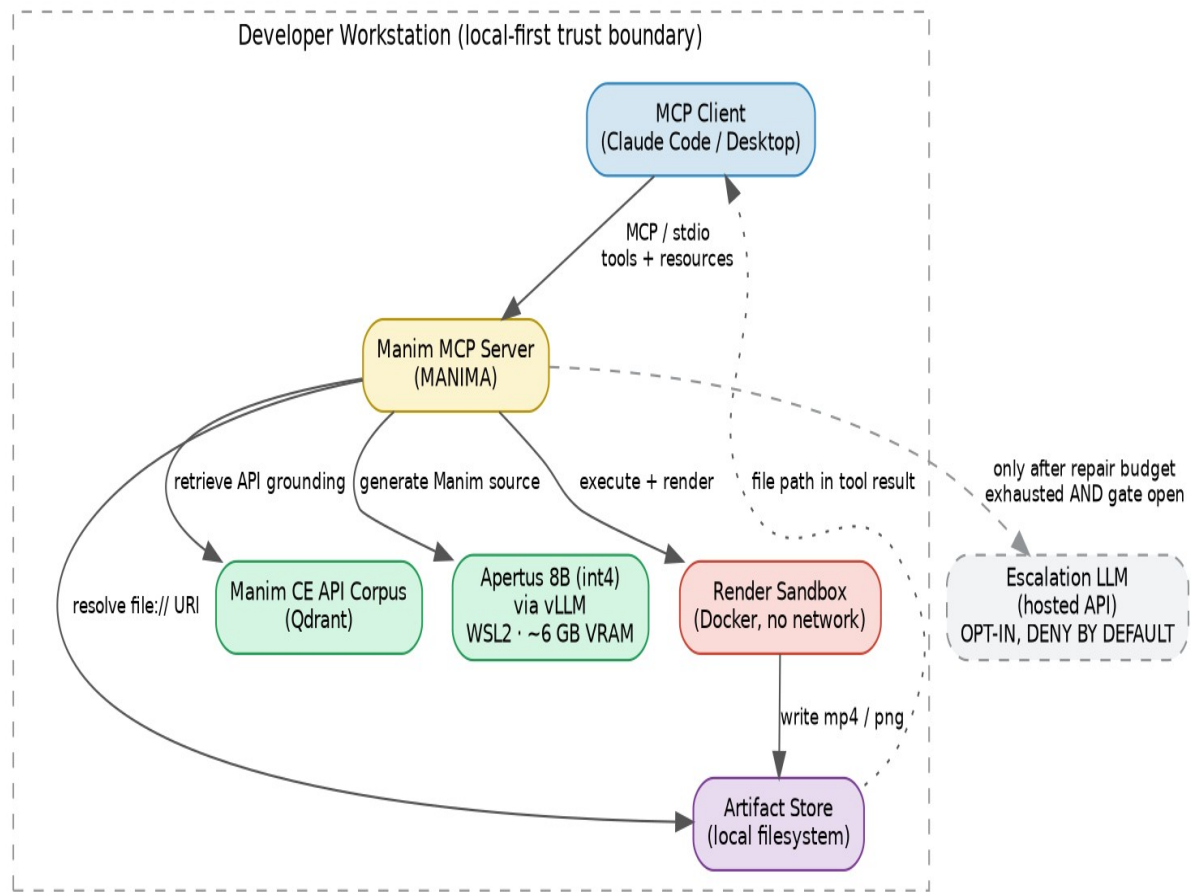


Figure 1 — System context. The dashed boundary is the local-first trust boundary; the escalation path crosses it and is denied by default.

3.1 External Interfaces

Interface	Direction	Protocol	Payload
MCP Client → MANIMA	Inbound	MCP over stdio	Tool calls (render, generate, status, result, cancel)
MANIMA → vLLM	Outbound (loopback)	HTTP (OpenAI-compatible)	Completion request with grounded prompt
MANIMA → Qdrant	Outbound (loopback)	HTTP	Vector query over Manim CE API corpus
MANIMA → Docker daemon	Outbound (local socket)	Docker API	Container create / start / kill
MANIMA → Escalation LLM	Outbound (internet)	HTTPS	GATED. Source + traceback. Denied by default.

4. Solution Strategy

Quality Goal	Architectural Approach
Security	Defence in depth: static AST validation rejects obvious hostile constructs before execution; a rootless Docker sandbox with no network, a read-only root filesystem, dropped capabilities, a seccomp profile, and CPU/memory/wall-clock limits contains whatever validation misses. The sandbox is the load-bearing control; validation is a fast-fail convenience, not a security boundary.
Sovereignty	Apertus 8B on vLLM is the default and only generator. Escalation is a distinct adapter behind a closed gate, emitting a receipt when used.
Correctness	A generate → validate → probe-render → repair loop. No source reaches the operator without having actually rendered at least once at low quality.
Responsiveness	Hexagonal core with an async job manager. Tool calls enqueue and return a job ID immediately; work proceeds on a worker.
Observability	Every job carries a structured trace: generator identity, attempt count, validation verdicts, tracebacks, escalation flag.

Design note. Placing generation inside the server inverts the usual capability gradient: a strong model (the MCP client) delegates code generation to a weaker local one. This is a deliberate trade of expected first-pass quality for sovereignty and zero marginal cost. The repair loop exists to make that trade survivable, and the dual tool surface exists so the trade can be declined per-call. See ADR-002.

5. Building Block View

5.1 Level 1 — Whitebox MANIMA

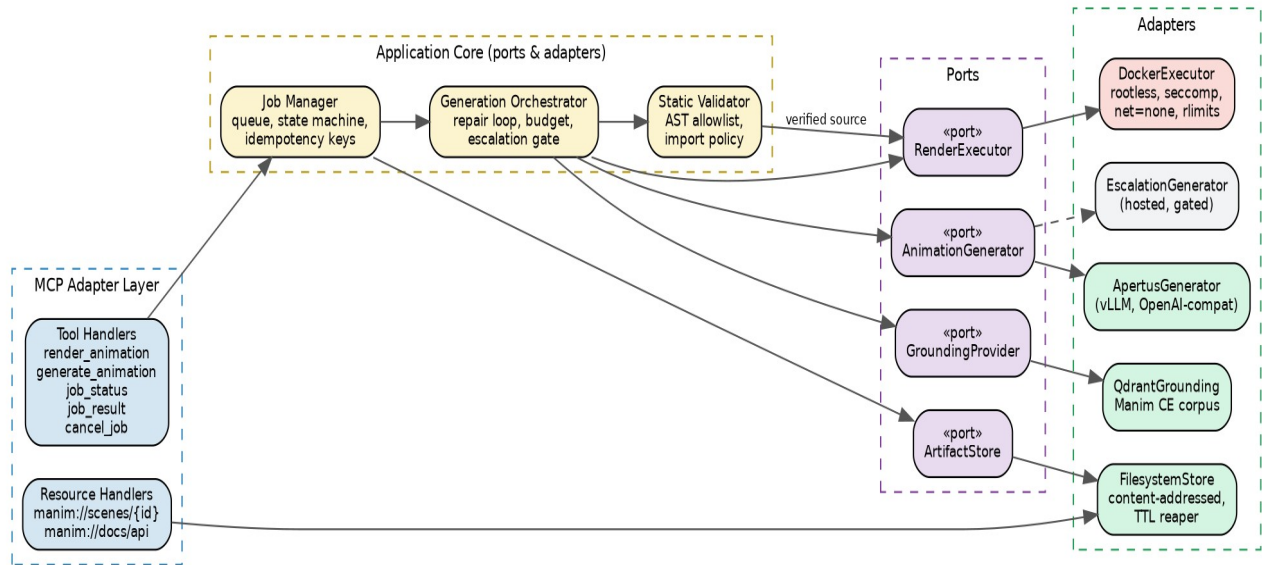


Figure 2 — Ports-and-adapters decomposition. The core depends only on ports; every external dependency is an adapter.

Building Block	Responsibility
Tool Handlers	Translate MCP tool calls into core commands. Validate argument schemas. Never contain domain logic.
Job Manager	Owns the job state machine, the work queue, idempotency keys, and TTL reaping of artifacts and logs.
Generation Orchestrator	Runs the repair loop. Owns the repair budget and the escalation gate decision.
Static Validator	AST-walks candidate source against an allowlist. Rejects disallowed imports, dunder access, and dynamic execution.
AnimationGenerator (port)	Contract: (prompt, grounding, repair_context?) → Manim source.
RenderExecutor (port)	Contract: (source, quality, limits) → artifact path traceback.
ArtifactStore (port)	Contract: content-addressed write, resolve to URI, TTL reap.
GroundingProvider (port)	Contract: (prompt) → ranked Manim CE API snippets.
ApertusGenerator	Adapter over vLLM's OpenAI-compatible endpoint. Default generator.
EscalationGenerator	Adapter over a hosted API. Gated, logged, off by default.
DockerExecutor	Rootless container per render. Enforces all sandbox controls.
FilesystemStore	Content-addressed store under a configured artifact root.
QdrantGrounding	Retrieves API snippets from an embedded Manim CE corpus.

5.2 Tool Surface

Five tools. The split between `render_animation` and `generate_animation` is the architectural hinge: it lets the caller choose who writes the code.

Tool	Arguments	Returns	Semantics
<code>render_animation</code>	<code>source</code> , <code>quality?</code> , <code>scene_name?</code>	<code>job_id</code>	Thin path. Caller supplies Manim source. Validated, sandboxed, rendered. No generator involved.
<code>generate_animation</code>	<code>prompt</code> , <code>quality?</code> , <code>repair_budget?</code> , <code>allow_escalation?</code>	<code>job_id</code>	Thick path. Apertus generates locally, then the repair loop runs.
<code>job_status</code>	<code>job_id</code>	<code>state</code> , <code>attempt</code> , <code>phase</code>	Cheap poll. Never blocks.
<code>job_result</code>	<code>job_id</code>	<code>artifact_uri</code> , <code>source</code> , <code>trace</code>	Terminal states only. Returns the file path, the final source, and the full attempt trace.
<code>cancel_job</code>	<code>job_id</code>	<code>ack</code>	Cooperative for queued/generating; SIGKILL to the container for rendering.

6. Runtime View

6.1 Generate-Verify-Repair

This is the load-bearing runtime scenario. It converts a generator of uncertain reliability into a component with a verified output contract: nothing is returned as successful that has not actually rendered.

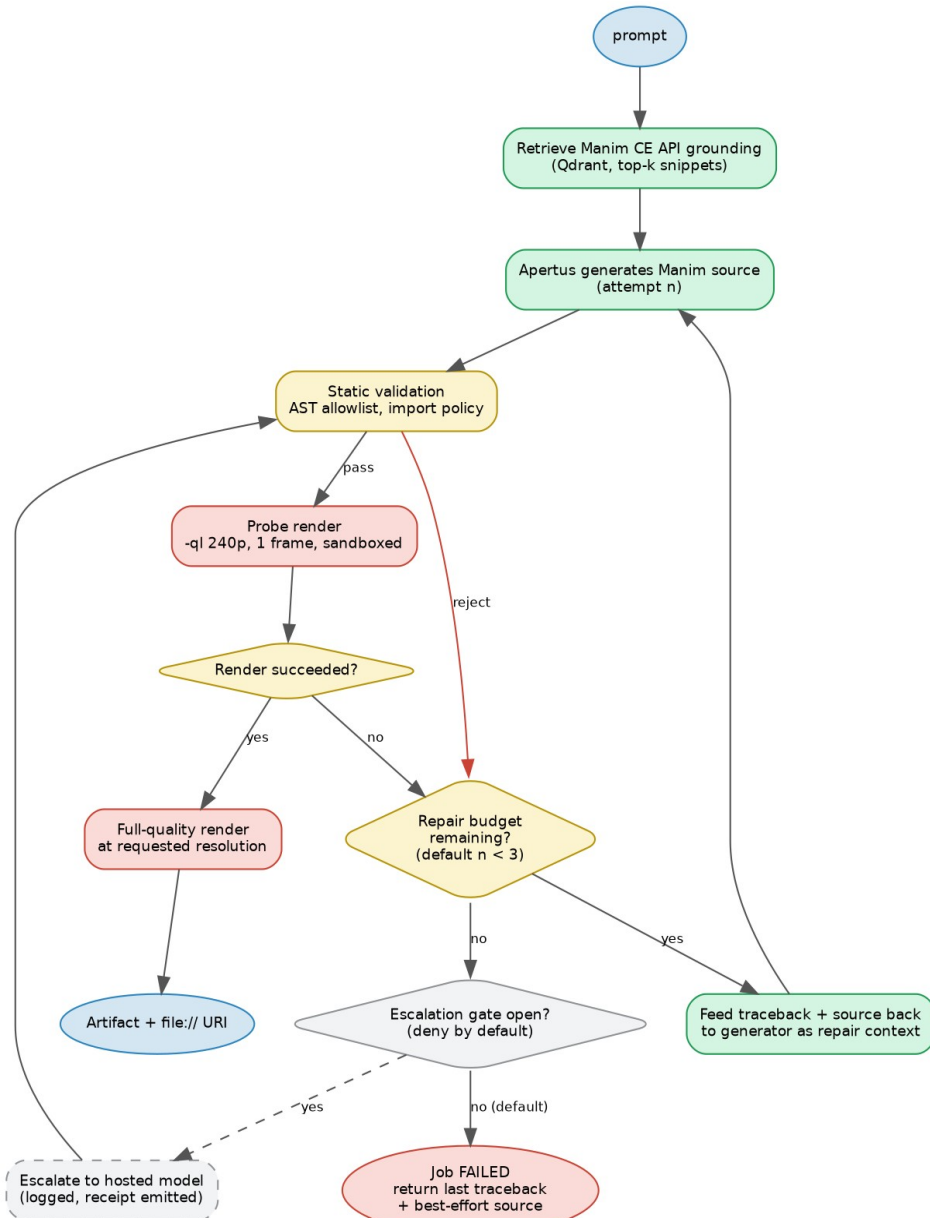


Figure 3 — The generate-verify-repair loop. The probe render is a cheap 240p single-frame execution used purely as a correctness oracle.

6.2 Why the probe render exists

A model cannot be trusted to know whether its own Manim code runs. **The probe render replaces that trust with evidence** at a cost of a few seconds. Rendering one frame at 240p exercises the entire import path, scene construction, and mobject instantiation — which is where essentially all generated-code failures live. Animation timing bugs survive the probe, but syntax errors, wrong-API calls, ManimGL/CE confusion, and missing imports do not.

The traceback from a failed probe is the highest-value input available to the repair attempt: it is specific, mechanical, and points at the exact construct the model got wrong. Feeding it back is what makes repair converge rather than merely resample.

6.3 Job Lifecycle

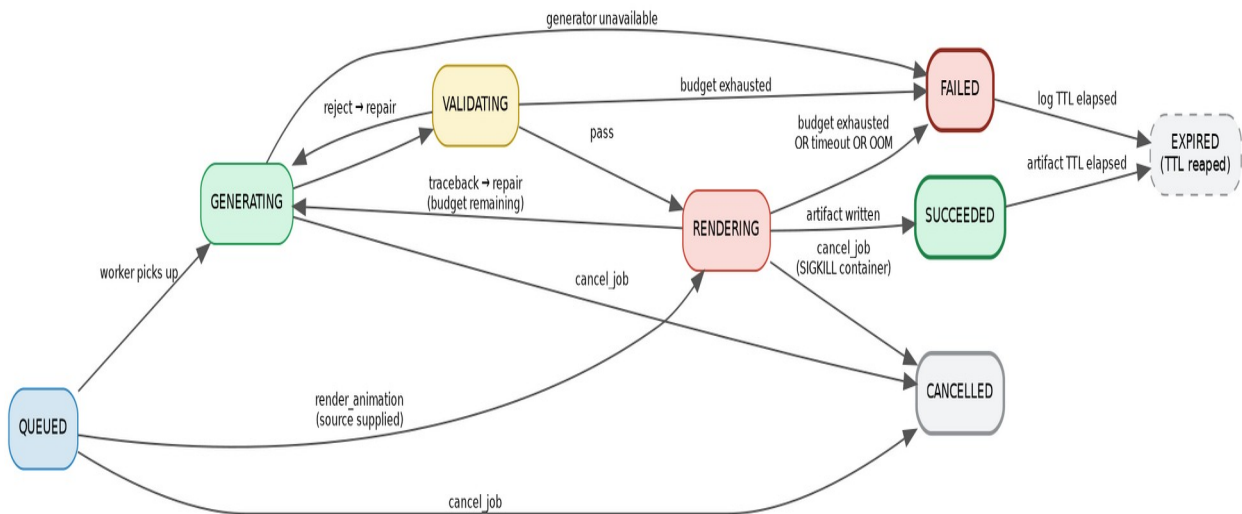


Figure 4 — Job state machine. **RENDERING → GENERATING** is the repair edge; it is the only backward transition.

7. Deployment View

A single Windows 11 desktop with an NVIDIA GPU (16 GB VRAM). Both vLLM and Docker require WSL2 on Windows, so the entire server side lives inside the WSL2 VM. Only the MCP client is Windows-native.

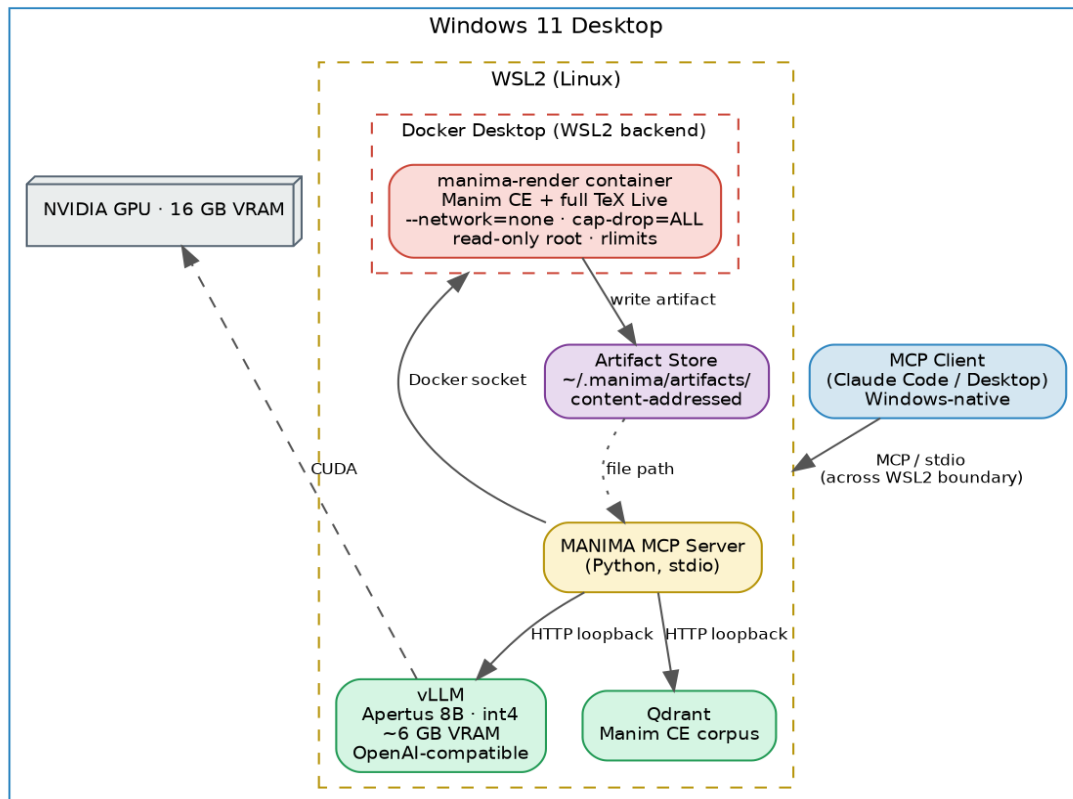


Figure 5 — Deployment topology. The WSL2 VM boundary is the primary containment; the container controls sit inside it as defence in depth.

Component	Placement	Notes
MCP Client	Windows-native	Claude Code or Desktop. Launches the server over stdio across the WSL2 boundary.
MANIMA server	WSL2	Python, stdio transport.
vLLM + Apertus 8B	WSL2	int4, ~6 GB VRAM. OpenAI-compatible endpoint on loopback. vLLM on Windows is WSL2-only.
Qdrant	WSL2	Manim CE grounding corpus.
Render container	WSL2 (Docker Desktop)	Manim CE + full TeX Live, ~4 GB image. One container per render.
Artifact store	WSL2 filesystem	Content-addressed, TTL-reaped. Keep inside the WSL2 filesystem — cross-boundary I/O to /mnt/c is slow enough to matter.

VRAM budget: Apertus 8B at int4 is roughly 6 GB, leaving around 10 GB for the KV cache. Grounded prompts are long, so that headroom is used, not wasted. Rendering is CPU-bound and does not contend with the GPU.

Sandbox Control	Setting	Rationale
Network	--network=none	A render has no legitimate need for network. Eliminates exfiltration and remote-payload fetch.
Root filesystem	read-only	Confines writes to an explicit output mount.
Capabilities	--cap-drop=ALL	Rendering needs none.
User	non-root, rootless daemon	A container escape lands as an unprivileged user, not root.
seccomp	restricted profile	Blocks syscall classes irrelevant to rendering.
Memory	hard cap (e.g. 4 GB)	Bounds OOM blast radius; a runaway scene fails its own job only.
CPU	quota	Prevents a render from starving inference.
Wall clock	hard timeout per phase	Probe and full render have separate budgets; timeout is a FAILED job, not a hang.
Mounts	output dir only, rw; source in, ro	No host filesystem visibility beyond the job's own working set.

8. Crosscutting Concepts

8.1 Grounding the Generator

The dominant failure mode for LLM-generated Manim is not incoherence — it is confidently emitting a valid-looking API that belongs to a different Manim version or to ManimGL entirely. ManimGL's `ShowCreation` versus CE's `Create` is the canonical example; there are dozens more.

Retrieval grounding attacks this directly. A corpus of Manim CE API signatures, docstrings, and canonical scene examples for the pinned version is embedded in Qdrant. Each generation request retrieves the top-k relevant snippets and injects them into the prompt. This pins the model to one version's reality rather than the union of everything it has read.

Expectation setting. Grounding plus repair is expected to lift a local model from unusable to workable on this task — not to parity with a frontier model. First-pass success rates should be measured, not assumed, and the escalation path exists precisely because the local path will sometimes not converge. Whether Apertus specifically clears the bar is an empirical question this architecture is designed to answer cheaply, not one to be settled by assertion.

8.2 Static Validation

An AST allowlist runs before any execution. It rejects imports outside a permitted set, attribute access to dunder, and calls to `eval`, `exec`, `compile`, and `__import__`.

Do not mistake this for a security boundary. AST allowlisting on Python is defeatable by a determined adversary and is not what stands between a hostile scene and the host — the sandbox is. Validation exists to fail fast and cheaply on the common case, and to produce a clean rejection message the repair loop can act on. Removing the sandbox because validation is present would be a serious error.

8.3 Escalation and Receipts

Escalation is deny-by-default at three independent levels: server configuration, per-call argument (`allow_escalation`), and exhaustion of the local repair budget. All three must be satisfied. When escalation fires, a receipt records the job ID, the model called, the token counts, and the reason the local path failed — so that the sovereignty property has an audit trail rather than a promise.

8.4 Artifact Lifecycle

Artifacts are content-addressed by a hash of (source, quality, `manim_version`). Identical requests return the existing artifact without re-rendering. A TTL reaper removes artifacts and job logs after a configured retention window. Tool results carry the path only — never the video bytes.

9. Architecture Decisions

ADR-001 — Sandbox all execution unconditionally

Field	Content
Status	Accepted
Context	Manim renders by executing Python. Both generated source and operator-supplied source are therefore executable payloads on the workstation.
Decision	Every render runs in a Docker container: no network, read-only root, all capabilities dropped, seccomp-restricted, non-root user, with memory, CPU, and wall-clock limits. This applies to <code>render_animation</code> as well as <code>generate_animation</code> .
Platform	Windows host, Docker Desktop on the WSL2 backend. Containers are Linux containers inside the WSL2 VM.
Consequences	(+) A broken or hostile scene cannot reach the Windows host filesystem or network. (+) Resource limits turn timeouts and OOMs into ordinary job failures. (-) Per-render container startup adds latency. (-) Docker Desktop becomes a hard dependency.
Rejected	Trusting operator-supplied source and sandboxing only generated source. Rejected: the operator's source is frequently pasted from elsewhere, so the trust distinction is illusory.

What WSL2 containment does and does not give you. On Linux-native Docker with a rootless daemon, the controls above are strong. On Windows via Docker Desktop, they are weaker than that, and this document will not pretend otherwise. Docker Desktop does not run rootless in the sense the Linux ecosystem means it; containers run inside a WSL2 VM whose own boundary is doing much of the real work. The honest formulation is: the WSL2 VM boundary is the primary containment, and the container controls are defence in depth inside it. That is adequate for the actual threat here — a small model emitting broken or careless code — and it is not a claim of hardened isolation against a determined adversary. If that stronger property is ever needed, the render sandbox must move to a Linux host or a hardened runtime such as gVisor or Firecracker.

ADR-002 — Generation inside the server, behind a port

Field	Content
Status	Accepted
Context	Manim source can be generated by the MCP client (a frontier model) or by a local model inside the server. The client is almost certainly the better code generator; the local model is the sovereign one.
Decision	Support both. <code>generate_animation</code> uses Apertus 8B (int4), served by vLLM under WSL2, as the default generator behind an <code>AnimationGenerator</code> port. <code>render_animation</code> lets the client supply source directly. Neither path is privileged in the architecture.
Model size	16 GB VRAM. Apertus 8B at int4 occupies roughly 6 GB, leaving ample headroom for the KV cache of a long grounded prompt. Apertus 70B does not fit at any usable quantisation (~40 GB at Q4). The 8B is therefore not a tuning choice but an arithmetic consequence of the hardware.
Runtime	vLLM, for concurrency and throughput under sustained load. On Windows this implies WSL2, which is acceptable because the Docker render sandbox requires WSL2 regardless.

Field	Content
Consequences	(+) The sovereignty/quality trade is made per-call by the caller, not baked in. (+) The port permits swapping the model or runtime without touching the core. (-) Two paths to maintain and test. (-) An 8B model is a weak generator for this task; see the caveat below.
Rejected	Thin renderer only. Rejected: forecloses local-first generation. Thick generator only. Rejected: forces a weaker generator on a caller who has a stronger one. Colibri as runtime. Rejected: it is a disk-streaming engine built for GLM-5.2 specifically, not a general runtime, and its throughput (~0.1–1 tok/s) is incompatible with an interactive repair loop.

Be clear-eyed about the 8B. An 8-billion-parameter model writing correct Manim CE is a hard ask. Small models lean hardest on the statistically dominant pattern in their training data, and for Manim that pattern is ManimGL and older CE versions — precisely the wrong answer. Expect poor first-pass success. Grounding (ADR-005) and the repair loop (ADR-003) are not polish here; they are the entire reason this path might work at all. The architecture is built so that if it does not, `render_animation` still does, and nothing else has to change.

ADR-003 — Bounded generate-verify-repair loop

Field	Content
Status	Accepted
Context	A local model's first-pass Manim output is expected to fail at a material rate, predominantly on API-version errors. Returning unverified source to the caller would push a rendering failure downstream, where the traceback is far less actionable.
Decision	Probe-render every candidate at 240p before accepting it. On traceback, feed the error and source back to the generator as repair context. Bound attempts (default 3). Only probe-verified source proceeds to full-quality render.
Consequences	(+) SUCCEEDED means the scene actually ran — a verified contract, not a hope. (+) Tracebacks make repair converge rather than resample blindly. (-) Worst case burns 3 generations plus 3 probe renders before failing. (-) Semantically wrong but syntactically valid animations still pass.
Rejected	Return generated source unverified. Rejected: shifts failure to the caller and wastes the cheapest available correctness signal.

ADR-004 — Asynchronous job protocol with path-referenced artifacts

Field	Content
Status	Accepted
Context	Renders take seconds to minutes; the repair loop multiplies this. Video artifacts are megabytes. MCP tool calls are synchronous and context-bearing.
Decision	All work is enqueued and returns a <code>job_id</code> immediately. Progress is polled via <code>job_status</code> . Terminal results are fetched via <code>job_result</code> , which returns a filesystem path to the artifact — never the bytes.
Consequences	(+) No tool call blocks. (+) Artifact size is decoupled from context budget. (+) Cancellation becomes expressible. (-) The client must poll. (-) The server must persist job state across the call boundary.
Rejected	Synchronous render with inline base64 video. Rejected: blocks the client for minutes and destroys the context window.

ADR-005 — Retrieval grounding against a pinned Manim CE corpus

Field	Content
Status	Accepted
Context	Manim's API has drifted substantially, and ManimGL forms are heavily represented in training data. An ungrounded model will emit deprecated or foreign APIs with high confidence.
Decision	Embed a corpus of API signatures, docstrings, and canonical examples for the pinned Manim CE version in Qdrant. Retrieve top-k per request and inject into the generation prompt. The Manim CE version is pinned in the render image and the corpus is built from that exact version.
Consequences	(+) Directly attacks the dominant failure mode. (+) Corpus rebuild is the upgrade path when Manim CE is bumped. (-) A corpus build step is added to the pipeline. (-) Grounding consumes prompt budget, which matters more on a smaller local model.
Rejected	Fine-tuning a model on Manim. Rejected: disproportionate effort, and retrieval is re-groundable in minutes when Manim CE releases.

ADR-006 — Full TeX Live in the render image

Field	Content
Status	Accepted
Context	Mathematical animation is the entire purpose. LaTeX typesetting is not an optional extra; a partial TeX install fails on exactly the symbols and packages that make a scene worth rendering.
Decision	Ship full TeX Live in the render image and accept the multi-gigabyte footprint. The image is built once and cached.
Consequences	(+) No class of LaTeX-dependent scene fails for want of a package. (-) Large image; slow first build. (-) A meaningfully larger surface inside the sandbox — which is the reason the sandbox controls in ADR-001 are not negotiable.
Rejected	TeX Live basic plus on-demand package installation. Rejected: requires network inside the sandbox, which contradicts ADR-001 outright.

10. Quality Requirements

10.1 Evaluation Scenarios

These are the scenarios that determine whether the architecture holds. The first is the one that decides whether the local-first default is viable at all.

ID	Scenario	Measure
Q-1	A benchmark suite of prompts spanning easy (plot a function), medium (transform between equations), and hard (3D surface with animated camera) is run against Apertus.	First-pass probe success rate; post-repair success rate; mean attempts consumed. This is the primary go/no-go signal for the local generator.
Q-2	A scene attempting <code>os.system</code> , <code>socket</code> , or file access outside its mount is submitted.	Rejected by validation OR contained by sandbox. Host state unchanged in both cases.
Q-3	The workstation's network is disabled entirely; <code>generate_animation</code> is called with the gate closed.	Job completes successfully. Confirms no hidden egress.
Q-4	A scene with an infinite loop in <code>construct()</code> is submitted.	Job reaches FAILED on wall-clock timeout; container is reaped; the server remains responsive.
Q-5	A 60-second 1080p60 scene is requested.	No tool call exceeds 2 s. Progress observable via <code>job_status</code> throughout.
Q-6	The same prompt is submitted twice.	Second call resolves from the content-addressed store without re-rendering.

Q-1 informs, but does not gate. Every other scenario tests a control this architecture already specifies. Q-1 tests an assumption it cannot specify away: whether Apertus 8B, grounded and repaired, is good enough at Manim CE to be a useful default generator. Run it early — but note that MANIMA remains a working animation tool whatever it returns, because `render_animation` does not depend on the answer. A poor Q-1 result reassigns `generate_animation` to experimental status; it does not stop the project. This is why the tool surface was split in two.

11. Risks and Technical Debt

Risk	Assessment	Mitigation
Apertus 8B is not good enough at Manim CE, even grounded and repaired.	The central risk, and a live one. Manim CE generation is narrow, version-sensitive, and unforgiving, and 8B is small. Small models lean on the dominant training-data pattern, which for Manim is ManimGL and older CE — the wrong answer. No repair budget fixes a model that does not know the API.	Measure with Q-1 early. The AnimationGenerator port makes the adapter swappable; the escalation path keeps the system useful regardless. If Q-1 is poor, make render_animation the documented primary path and mark generation experimental. This is a graceful degradation, not a project failure.
Repair loop fails to converge and merely burns budget.	Plausible if the model's errors are conceptual rather than mechanical. Repair works on tracebacks; it cannot fix a model that does not know Create exists. More likely at 8B than at larger sizes.	Instrument attempt-over-attempt improvement. If attempt 3 is no better than attempt 1, cut the budget to 1 and escalate sooner — the budget is buying nothing.
Sandbox containment is weaker on WSL2 than on Linux-native.	Real. Docker Desktop is not rootless in the Linux sense; the WSL2 VM boundary is doing most of the containment work. Full TeX Live is a large attack surface and TeX has a shell-escape history.	Disable TeX shell-escape explicitly; drop capabilities; no network; read-only root. Accept that this bounds accidental and careless damage, not a determined adversary. If stronger isolation is ever required, move the sandbox to a Linux host or gVisor/Firecracker. Do not overstate the current property.
Probe render passes, full render diverges.	Possible: 240p single-frame does not exercise timing, camera paths, or long-running animation logic.	Treat the probe as a syntax and API oracle only. Do not claim it validates semantics; report it honestly in job_result.
Docker daemon dependency.	A hard runtime dependency on a component outside the server.	Fail fast and loudly at startup if the daemon is unreachable. Do not fall back to unsandboxed execution under any circumstance.
Content-addressed cache serves stale artifacts after a Manim upgrade.	Real, and silent if the hash omits the version.	Include manim_version in the hash input. Rebuild the grounding corpus in the same step as any Manim CE bump.

12. Glossary

Term	Definition
Apertus	Open-weight Swiss large language model. The 8B variant at int4 is the default local generator in this architecture.
vLLM	Inference server hosting Apertus. Exposes an OpenAI-compatible endpoint. Requires WSL2 on Windows.
WSL2	Windows Subsystem for Linux v2. Hosts the server, vLLM, Qdrant, and the Docker render sandbox. Its VM boundary is the primary containment for untrusted render code.
Escalation gate	The three-condition control (server config, per-call flag, exhausted local budget) that must be fully satisfied before any hosted model is contacted.
Manim CE	Manim Community Edition — the maintained fork. Distinct from ManimGL, whose API differs and whose forms contaminate generated code.
MCP	Model Context Protocol. The client-server protocol over which tools and resources are exposed.
Probe render	A 240p single-frame sandboxed render used purely as a correctness oracle for generated source.
Repair budget	The bounded number of generate-and-reprobe attempts permitted before a job fails or escalates.
Receipt	A structured record emitted whenever escalation fires, recording model, tokens, and cause.